

Guide d'exploration - Patron Décorateur

Ajouter des responsabilités sans modifier les classes existantes

Support progressif du TP4 - Design Patterns & Frameworks

Objectif. Comprendre pourquoi on évite d'ajouter des traitements transverses directement dans les adaptateurs, puis savoir utiliser le patron **Decorator** pour enrichir une passerelle tout en conservant le contrat **PaymentGateway**.

Le guide suit la continuité des TP1, TP2 et TP3 : les passerelles sont déjà adaptées, sélectionnées et créées par boutique ; il reste maintenant à les enrichir proprement.

1. Comment lire ce guide

Ce document n'est pas une fiche de définitions à mémoriser.

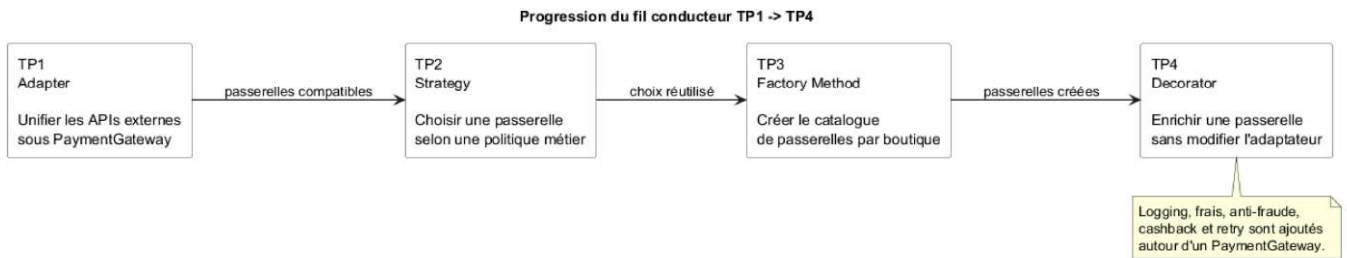
Il doit être lu comme une exploration de conception : on part d'un besoin métier, on observe une solution naïve, puis on identifie progressivement pourquoi **Decorator** est plus adapté.

À chaque étape, posez-vous trois questions :

- quelle classe reçoit une nouvelle responsabilité ?
- le code client doit-il connaître cette responsabilité ?
- que faut-il modifier si l'on ajoute un nouveau comportement ?

La progression suivie est la suivante :

Étape	Idée étudiée	Question principale
1	Modification directe des adaptateurs	Pourquoi cette solution devient-elle fragile ?
2	Interface commune	Pourquoi le décorateur doit-il rester un PaymentGateway ?
3	Décorateur abstrait	Comment déléguer sans dupliquer ?
4	Décorateurs concrets	Comment ajouter une responsabilité par classe ?
5	Chaîne de décorateurs	Pourquoi l'ordre de composition compte-t-il ?
6	Application au TP4	Où placer l'enrichissement dans le flux du checkout ?



Source PlantUML : `../diagrams/figure4_progression_tp1_tp4.puml`

2. Étape 1 - Observer la solution naïve

Le TP4 part d'une question simple :

Comment ajouter un comportement autour d'une passerelle de paiement sans modifier sa classe ?

Une première solution consiste à modifier directement chaque adaptateur :

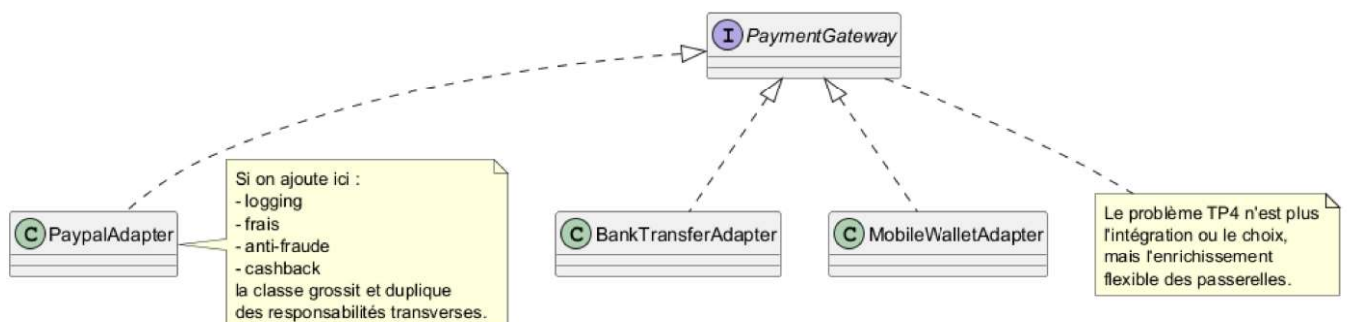
```

public class PayPalAdapter implements PaymentGateway {
    @Override
    public PaymentResult pay(CheckoutContext context) {
        System.out.println("Début paiement PayPal");
        // contrôle anti-fraude
        // appel de PayPalSdk
        // calcul du cashback
        System.out.println("Fin paiement PayPal");
    }
}
  
```

Cette solution semble rapide, mais elle pose plusieurs problèmes :

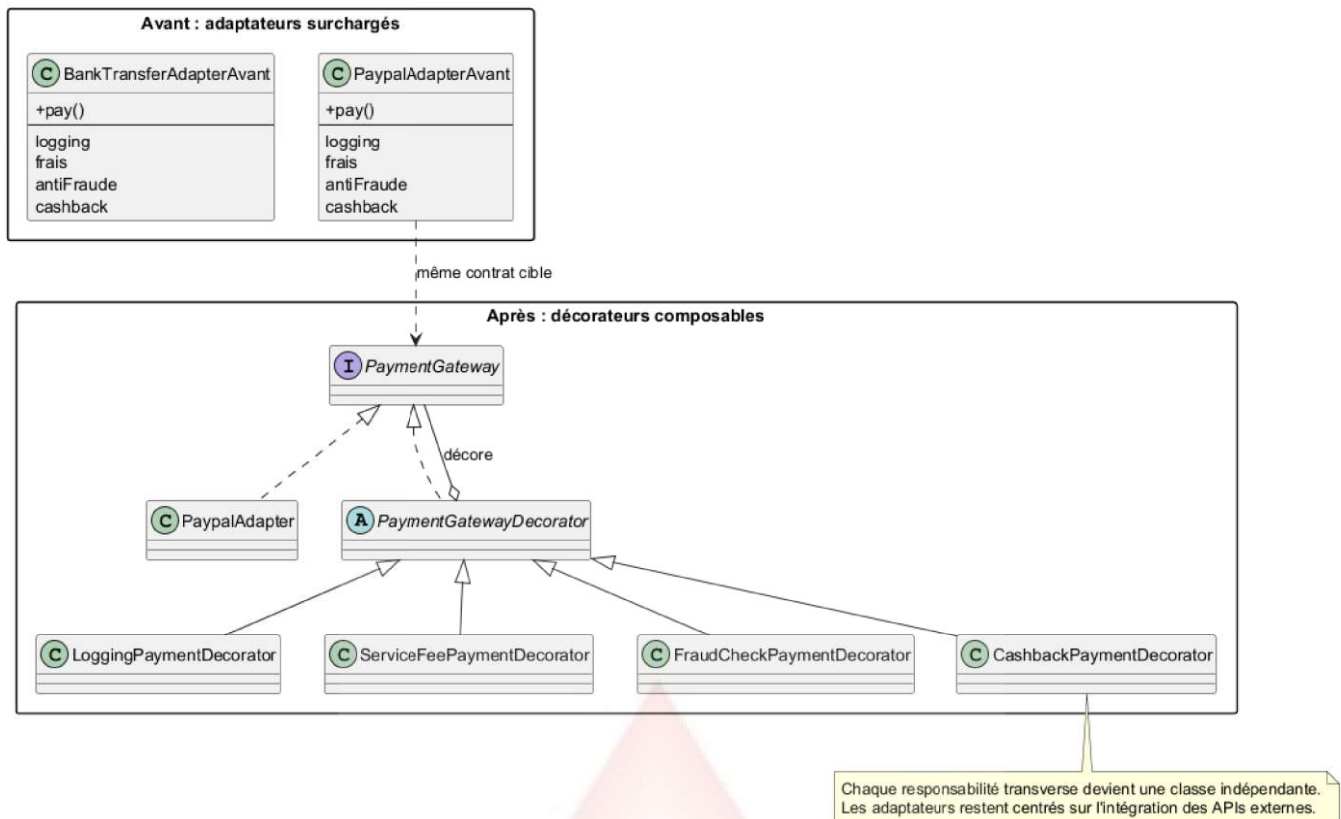
- les adaptateurs ne font plus seulement de l'adaptation ;
- les mêmes traitements sont copiés dans plusieurs classes ;
- les combinaisons de comportements deviennent difficiles à gérer ;
- le code client risque de connaître trop de détails ;
- l'ajout d'un nouveau comportement impose de modifier du code existant.

TP4 - Problème sans Décorateur



Source PlantUML : `../diagrams/figure1_problem_without_decorator.puml`

Avant / après : déplacer les responsabilités transverses

Source PlantUML : `../diagrams/figure5_before_after_decorator.puml`

Activité d'observation

Répondez avant de lire la suite :

- Que faut-il modifier si l'on ajoute un comportement de chiffrement ?
- Que faut-il modifier si seule la boutique Premium doit avoir du cashback ?
- Que faut-il modifier si la boutique Europe veut uniquement retry + logging ?
- Est-ce vraiment à `CheckoutService` de connaître ces détails ?

3. Étape 2 - Garder une interface commune

Le point central du patron `Decorator` est la substituabilité.

Un décorateur doit pouvoir être utilisé exactement comme l'objet qu'il enrichit.

Dans le TP4, l'interface commune est :


```

public interface PaymentGateway {
    boolean supports(CheckoutContext context);
    double estimateFee(CheckoutContext context);
    int estimateConfirmationDelayMinutes(CheckoutContext context);
    int getSecurityScore();
    PaymentResult pay(CheckoutContext context);
    String getProviderName();
}

```

Un adaptateur concret, comme `PaypalAdapter`, implémente cette interface.

Un décorateur, comme `LoggingPaymentDecorator`, l'implémente aussi.

Cela signifie que `CheckoutService` peut recevoir indifféremment :

```
PaymentGateway gateway = new PaypalAdapter(new PaypalSdk());
```

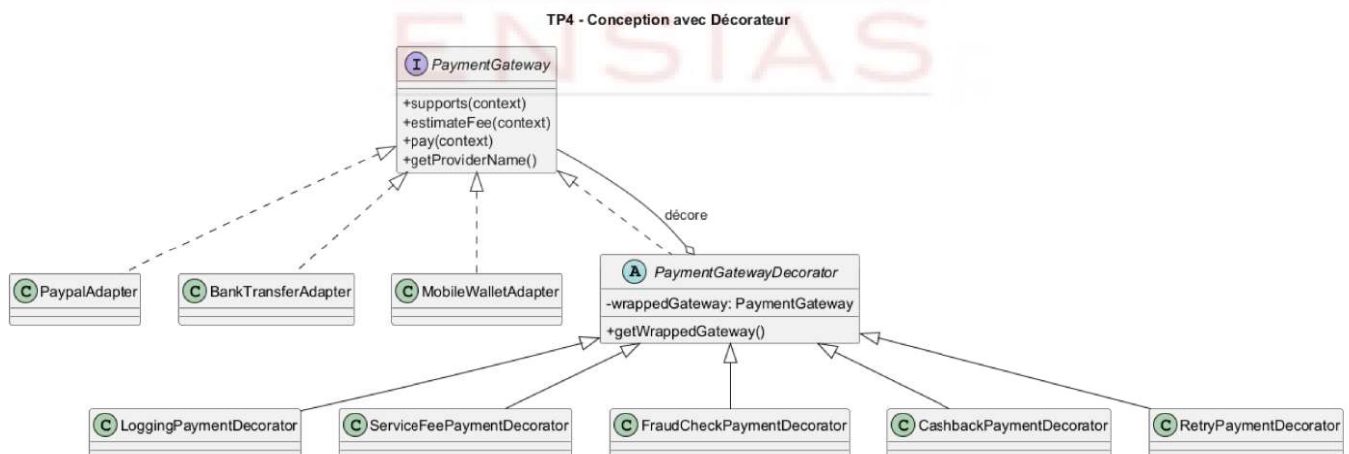
ou :

```

PaymentGateway gateway =
    new LoggingPaymentDecorator(
        new PaypalAdapter(new PaypalSdk())
    );

```

Dans les deux cas, le type manipulé reste `PaymentGateway`.



Source PlantUML : `../diagrams/figure2_decorator_design.puml`

4. Étape 3 - Introduire le décorateur abstrait

Le décorateur abstrait évite de recopier la délégation dans chaque décorateur concret.

Il contient une référence vers l'objet décoré :

```
public abstract class PaymentGatewayDecorator implements PaymentGateway {  
    private final PaymentGateway wrappedGateway;  
  
    protected PaymentGatewayDecorator(PaymentGateway wrappedGateway) {  
        this.wrappedGateway = wrappedGateway;  
    }  
}
```

Par défaut, il délègue les opérations à `wrappedGateway`.

Chaque sous-classe ne redéfinit que ce qu'elle veut enrichir.

Par exemple :

- `LoggingPaymentDecorator` redéfinit surtout `pay()` ;
- `ServiceFeePaymentDecorator` redéfinit `estimateFee()` et enrichit le message de succès ;
- `FraudCheckPaymentDecorator` redéfinit `supports()`, `pay()` et `getSecurityScore()` ;
- `CashbackPaymentDecorator` redéfinit `pay()` ;
- `RetryPaymentDecorator` redéfinit `pay()`.

5. Étape 4 - Ajouter une responsabilité par décorateur

Un bon décorateur doit rester petit.

Il ajoute une seule responsabilité clairement identifiable.

Logging

Le logging entoure l'appel réel :

```
public PaymentResult pay(CheckoutContext context) {  
    System.out.println("[LOG] Début paiement " + getProviderName());  
    PaymentResult result = wrapped().pay(context);  
    System.out.println("[LOG] Fin paiement " + getProviderName());  
    return result;  
}
```

Frais de service

Les frais modifient l'estimation du coût :

```
public double estimateFee(CheckoutContext context) {  
    return wrapped().estimateFee(context) + additionalFee;  
}
```

Anti-fraude

L'anti-fraude peut refuser le support d'un paiement :

```
public boolean supports(CheckoutContext context) {  
    return wrapped().supports(context)  
        && context.getOrder().getAmount() <= suspiciousAmountLimit;  
}
```

Cashback

Le cashback ne change pas la compatibilité de la passerelle.

Il enrichit le résultat lorsque le paiement réussit.

6. Étape 5 - Composer une chaîne de décorateurs

Les décorateurs peuvent être combinés :

```
PaymentGateway gateway =  
    new LoggingPaymentDecorator(  
        new CashbackPaymentDecorator(  
            new FraudCheckPaymentDecorator(  
                new ServiceFeePaymentDecorator(  
                    new PayPalAdapter(new PayPalSdk()),  
                    3.00,  
                    0.003  
                ),  
                10000.00  
            ),  
            0.01  
        )  
    );
```

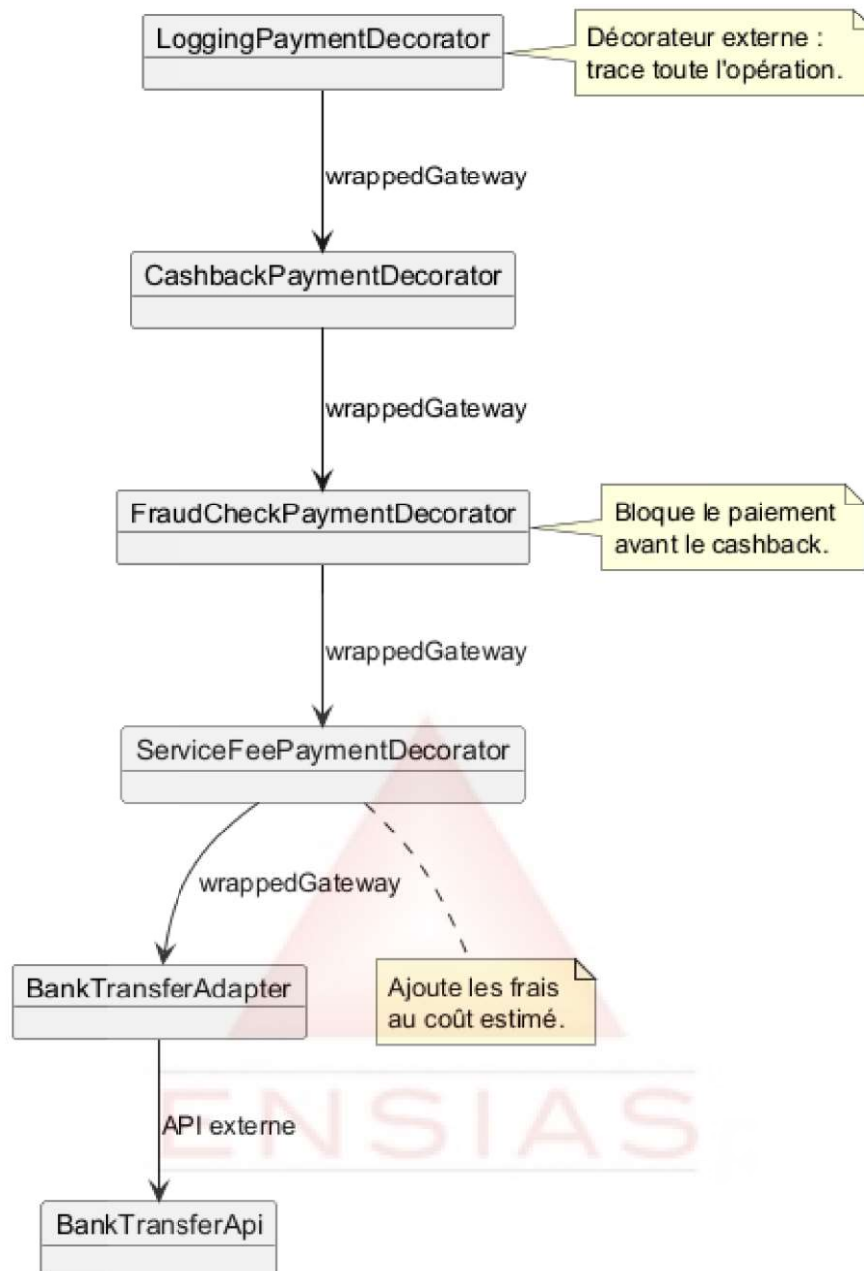
La lecture se fait de l'extérieur vers l'intérieur pour comprendre l'enveloppe, mais l'exécution descend d'abord vers l'objet décoré puis remonte avec les enrichissements.

L'ordre peut changer le comportement.

Si le cashback est placé avant l'anti-fraude, le code peut devenir ambigu : annonce-t-on un cashback pour un paiement qui sera ensuite bloqué ?

Dans le TP4, on place donc l'anti-fraude avant le cashback dans la chaîne métier.

Chaîne de décorateurs - Boutique Premium



Source PlantUML : `../diagrams/figure6_premium_decorator_chain.puml`

7. Étape 6 - Relier Decorator aux patrons précédents

Les patrons utilisés dans les TP ont chacun une responsabilité précise :

Patron	Question résolue dans le projet
Adapter	Comment rendre compatibles plusieurs APIs externes ?
Strategy	Quelle passerelle choisir selon le contexte ?
Factory Method	Quelles passerelles créer pour une boutique ?
Decorator	Quels comportements ajouter autour d'une passerelle ?

Decorator ne remplace donc pas **Factory Method**.

La fabrique crée le catalogue.

Le décorateur enrichit les éléments du catalogue.

La stratégie choisit ensuite parmi les passerelles disponibles, qu'elles soient décorées ou non.

8. Application directe au TP4

Dans le code du TP4, l'enrichissement se fait dans **CheckoutApplication** :

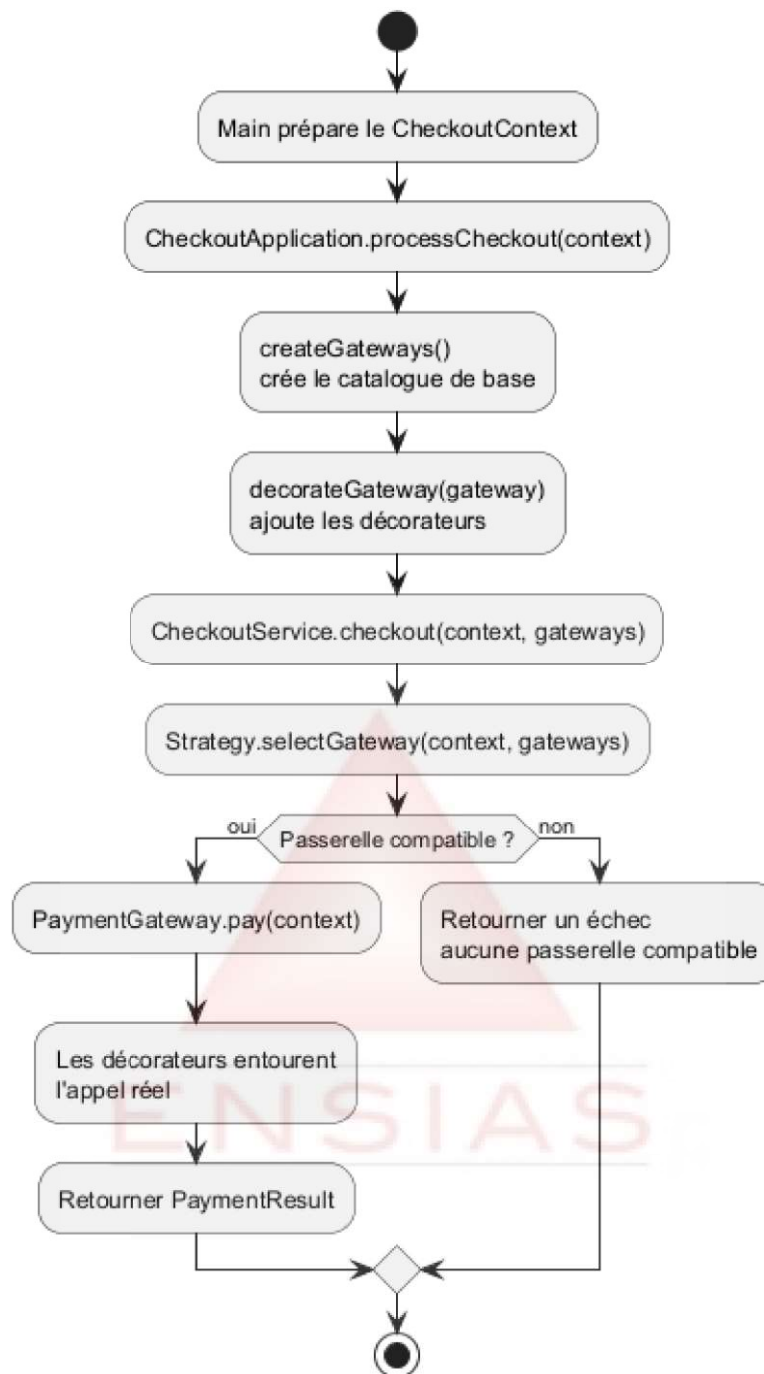
```
public final List<PaymentGateway> createAvailableGateways() {  
    return createGateways().stream()  
        .map(this::decorateGateway)  
        .toList();  
}
```

Chaque boutique peut ensuite redéfinir :

```
protected PaymentGateway decorateGateway(PaymentGateway gateway) {  
    return new LoggingPaymentDecorator(  
        new ServiceFeePaymentDecorator(gateway, 1.50, 0.002)  
    );  
}
```

Cette organisation garde une séparation claire :

- **createGateways()** crée les passerelles ;
- **decorateGateway()** ajoute les responsabilités ;
- **CheckoutService** exécute le checkout ;
- les stratégies choisissent une passerelle compatible.

Pipeline TP4 : créer, décorer, choisir, payer

Source PlantUML : `../diagrams/figure7_checkout_pipeline.puml`

9. Erreurs fréquentes à éviter

Évitez les erreurs suivantes :

- modifier directement les adaptateurs pour ajouter les responsabilités transverses ;
- créer une classe énorme du type `SecureLoggedCashbackPaypalAdapter` ;
- placer la logique des décorateurs dans `CheckoutService` ;
- utiliser des conditions métier partout dans `Main` ;
- faire un décorateur qui n'implémente pas `PaymentGateway` ;

- oublier de déléguer au composant décoré.

10. Questions de synthèse

Répondez en quelques phrases :

- Pourquoi le décorateur abstrait contient-il une référence vers `PaymentGateway` ?
- Pourquoi `CheckoutService` peut-il rester inchangé ?
- Pourquoi `ServiceFeePaymentDecorator` doit-il modifier `estimateFee()` ?
- Pourquoi `FraudCheckPaymentDecorator` peut-il modifier `supports()` ?
- Pourquoi la composition des décorateurs appartient-elle plutôt aux boutiques qu'aux adaptateurs ?

11. Idée clé

Le patron `Decorator` permet d'ajouter des responsabilités à un objet sans modifier sa classe.

Dans ce TP, il permet d'enrichir les passerelles de paiement tout en gardant l'architecture construite progressivement dans TP1, TP2 et TP3.

